

Rutaflow — Documentación de Replicación del Sistema

Versión: 2026-09-10

Propósito: Guía técnica para entender, desplegar y replicar el ecosistema Rutaflow completo.

1. Resumen ejecutivo

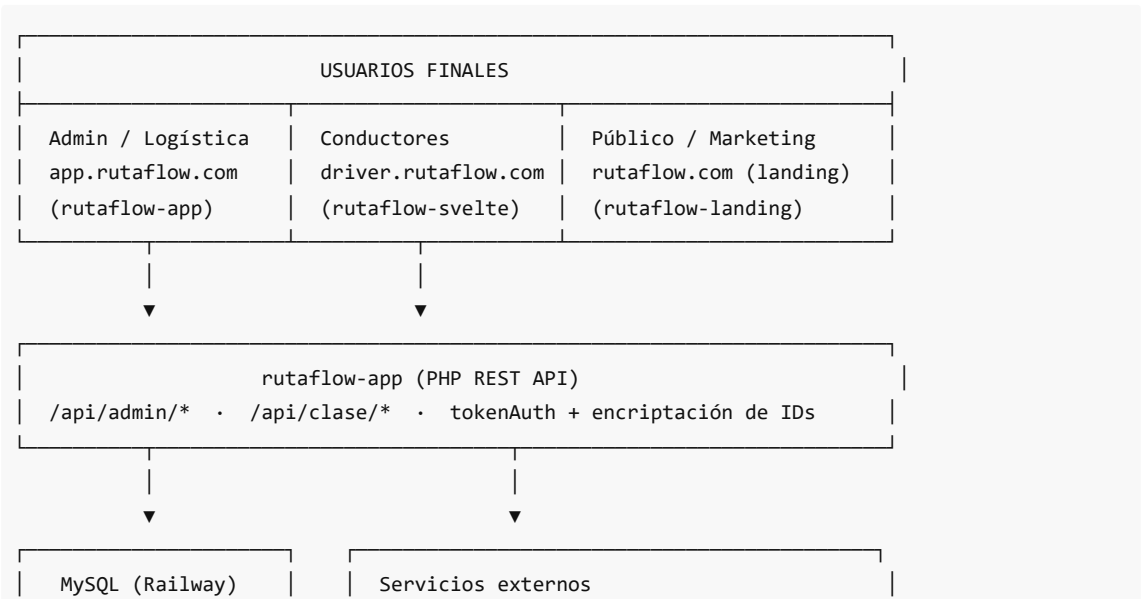
Rutaflow es una plataforma **multi-tenant** de gestión logística de rutas y entregas. Permite a empresas (enterprises) planificar plantillas de ruta, asignar conductores y vehículos, ejecutar rutas en campo con app móvil, capturar evidencias, checklists, firmas y generar reportes operativos.

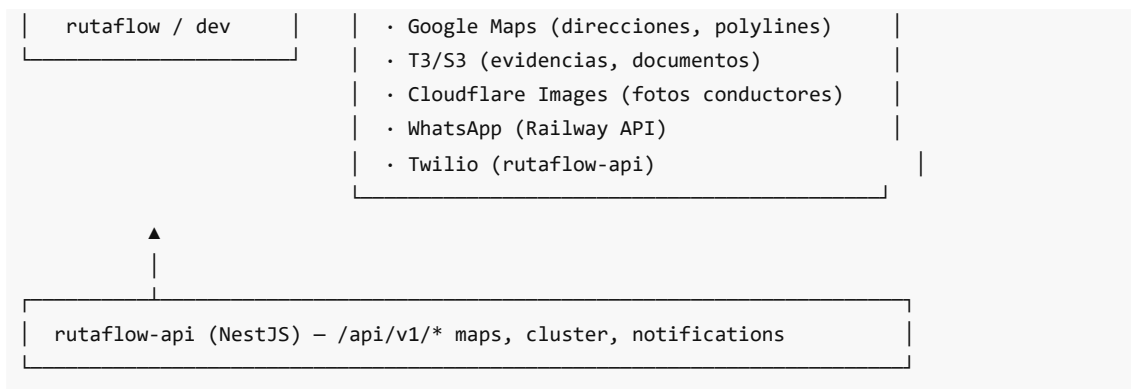
El sistema actual está **distribuido en cuatro repositorios/proyectos** dentro del monorepo `rutaflow/` :

Proyecto	Rol	Stack principal
rutaflow-app	Panel admin + API PHP legacy (núcleo operativo)	PHP 8+, MySQL, AngularJS/jQuery, Bootstrap
rutaflow-svelte	App móvil conductores (web + APK/iOS)	SvelteKit 2, Ionic 7, Capacitor 5, Node adapter
rutaflow-api	API NestJS complementaria (maps, clustering, notificaciones)	NestJS 10, Prisma, TypeScript
rutaflow-landing	Sitio marketing	Next.js

Fuente de verdad de datos: MySQL compartida. La mayoría de la lógica de negocio vive en **rutaflow-app** (PHP). La app móvil consume endpoints PHP; NestJS cubre funciones específicas (optimización, WhatsApp, etc.).

2. Arquitectura general





Entornos

Entorno	Admin	App conductores	Notas
Producción	https://app.rutaflow.com	https://driver.rutaflow.com	Datos reales
Desarrollo	https://dev.rutaflow.com	https://dev-driver.rutaflow.com	BD dev separada
Local	localhost:8000	localhost:5173 (Vite)	Ver DEPLOYMENT.md

Zona horaria operativa: UTC-6 (México). Los filtros de fecha en rutas y reportes aplican `DATE_ADD(..., INTERVAL -6 HOUR)` o `COALESCE(date_started, date_start)` para alinear día calendario con inicio real.

3. Stack tecnológico detallado

3.1 rutaflow-app (Admin + API PHP)

- **Runtime:** PHP 8.2+
- **Base de datos:** MySQL vía ADOdb (`api/framework/adodb/`)
- **Frontend admin:** AngularJS 1.x, jQuery, Bootstrap 3, DataTables, Google Maps JS
- **Estructura clave:**
 - `api/admin/` — Endpoints REST por módulo (route, driver, event, report, dashboard, app)
 - `api/clase/` — Clases modelo con SQL embebido (Route, Event, Driver, Stop, etc.)
 - `api/framework/` — Funciones.php, encriptación, S3Helper, EnvLoader
 - `sys/` — Panel admin (layout PHP + assets JS)
 - `api/db/` — Dump inicial + migraciones SQL

3.2 rutaflow-svelte (App conductores)

- **Framework:** SvelteKit 2 + Vite 5 + Svelte 4
- **UI móvil:** Ionic Core 7 + ionic-svelte
- **Nativo:** Capacitor 5 (Android/iOS)
- **Adapter:** `@sveltejs/adapter-node` (servidor Node en producción)
- **Persistencia local:** `@macfja/svelte-persistent-store` , Capacitor Preferences
- **Funcionalidades:** Login, lista rutas, detalle ruta, paradas, checklist, evidencias, firma, gastos, geolocalización

Config API: `src/hooks.js` define `DATABASE_URL` (URL base del backend PHP). En prod apunta a `https://app.rutaflow.com/` .

3.3 rutaflow-api (NestJS)

- **Framework:** NestJS 10
- **ORM:** Prisma 5 (`src/prisma/schema.prisma`)
- **Módulos:** user, driver, enterprise, route, stop, vehicle, maps, cluster
- **Integraciones:** Google Maps Services JS, node-kmeans, Twilio, JWT/bcrypt
- **Deploy típico:** Railway (`rutaflow-api-production.up.railway.app`)

3.4 rutaflow-landing

- Next.js con CI en GitHub Actions (`.github/workflows/nextjs.yml`)

3.5 Infraestructura y almacenamiento

- **BD:** MySQL en Railway (hosts/puertos por entorno en `.env`)
- **Archivos:** T3 Storage API (compatible S3) — buckets separados prod/dev
- **Imágenes conductores:** Cloudflare Images API
- **CDN:** Cloudflare delante de dominios (purgar caché tras deploy de CSS/JS)

4. Modelo de datos (entidades principales)

***Nota:** El schema Prisma en `rutaflow-api` es referencia parcial. Producción puede tener campos/enums adicionales (`date_started` , `checklist-pending` , `finished`). Usar `api/db/db_170925.sql` + migraciones en `api/db/migrations/` como fuente completa.*

4.1 Jerarquía multi-tenant

```
Enterprise (empresa)
├─ User (admin, manager, logistics, driver, super, external)
├─ Driver (operador vinculado a User)
├─ Vehicle (unidad)
├─ Client (cliente/contrato con checklist propio)
│   └─ Stop (paradas/direcciones)
│       └─ Checklist (items pre-ruta)
├─ RouteTemplate (plantilla reutilizable)
│   └─ EventTemplate (paradas ordenadas)
└─ Route (ruta operativa del día)
    └─ Event (parada ejecutable = stop + estado entrega)
        └─ Evidence (fotos/documentos por parada)
            └─ ChecklistEvent (checklist instanciado por ruta)
```

4.2 Tablas clave

Tabla	Descripción
enterprise	Tenant. Tiene settings JSON con flags (stops_wait_approval, etc.)
user	Credenciales, tipo, token sesión, id_enterprise
driver	Extensión operador: docs, status, foto, enterprise
vehicle	Unidad: placa, seguro, status, enterprise
client	Cliente logístico con tiempos pickup/delivery

stop	Dirección geocodificada (lat/lon), tipo, archivo referencia
route_template	Plantilla con polyline, distancia, paradas
route	Ruta asignada: driver, vehicle, fechas, status, polyline
event	Instancia de parada en ruta: orden, ETA, deliver_status, firmas
evidence	Imagen/archivo ligado a event
checklist / checklist_event	Items pre-ruta y su cumplimiento
report	Definiciones SQL dinámicas para reportes admin
incidence	Incidencias operativas ligadas a rutas
cms_menu	Menú dinámico del CRUD admin

4.3 Campos de fecha en rutas (regla crítica)

Campo	Significado
date_start	Fecha/hora planificada al crear la ruta
date_started	Fecha/hora real de inicio (checklist completado / salida)
date_end	Fin planificado o real según flujo

Regla de negocio: Para listados del día, KPIs y reportes, usar:

```
COALESCE(r.date_started, r.date_start)
```

Con ajuste `-6 HOUR` para zona México. Rutas activas (`enroute` , `checklist` , etc.) pueden aparecer aunque su fecha calendario no coincida.

5. Reglas de negocio

5.1 Tipos de usuario (`user.type`)

Valor	Rol
super	Superadmin global
admin	Admin de enterprise
manager	Gestión operativa
logistics	Logística / despacho
driver	Conductor (app móvil)
external	Acceso limitado externo

El panel admin filtra datos por `id_enterprise` seleccionado (Enterprise Manager en frontend).

5.2 Ciclo de vida de una ruta (route.status)

Status	Significado	Visible como "activa"
pending	Creada, no iniciada	No (pendiente)
checklist	Conductor completando checklist	Sí
checklist-pending	Checklist con items obligatorios sin aprobar	Sí
enroute	En curso / entregando paradas	Sí
paused	Pausada temporalmente	Sí
completed	Completada (todas paradas)	No
finished	Finalizada manualmente ("Terminar")	No
cancelled	Cancelada	No

Transiciones típicas (app móvil):

1. Admin crea ruta → pending
2. Conductor abre ruta → checklist o checklist-pending
3. Admin aprueba checklist (si aplica) → enroute
4. Paradas completadas → completed
5. Botón "Terminar" → finished

Estados terminales: completed , finished , cancelled — excluidos de disponibilidad driver/vehicle.

5.3 Sincronización driver/vehicle con ruta

Funciones en api/framework/Funciones.php :

- mapRouteStatusToEntityStatus() — Mapea status ruta → status entidad
- syncEntityStatusWithActiveRoutes() — Actualiza driver/vehicle al cambiar ruta
- isDriverAssignable() — Driver en workshop no es asignable

Status ruta	Status driver/vehicle resultante
pending	enroute
checklist, checklist-pending, enroute, paused	Igual que ruta
completed, finished, cancelled	available

Status manuales (operaciones admin): available , workshop , nodocs , paused

5.4 Eventos / paradas (event)

Campo	Valores	Descripción
status	pending, route, issue, service, completed	Estado operativo parada
deliver_status	delivered, partialdeliver, notdelivered	Resultado entrega

priority_status	urgent, normal, noturgent	Prioridad
-----------------	---------------------------	-----------

Cada `event` vincula `id_route` + `id_stop` + orden (`pos`). Al completar parada se registran coordenadas (`ev_lat` , `ev_lon`), tiempos y comentarios.

5.5 Checklist pre-ruta

1. Al **crear ruta** (`create_route.php`), se copian items de `checklist` del `id_client` a `checklist_event`
2. Conductor completa items en app (fotos opcionales/obligatorias según config)
3. Si hay items obligatorios sin completar → ruta pasa a `checklist-pending`
4. Admin puede **aprobar checklist** desde panel → pasa a `enroute`
5. `date_started` se setea al completar/aprobar checklist

5.6 Evidencias

- Tabla `evidence` : ligada a `event` , campo `type` , `approved` (0/1)
- Upload a bucket S3/T3 vía endpoints PHP (`upload_document.php` , etc.)
- Admin revisa y aprueba evidencias desde panel de ruta actual

5.7 Configuración por enterprise (`enterprise.settings`)

Flags JSON interpretados en frontend (ej. `current_routes.js`):

Alias	Efecto
<code>stops_wait_approval</code>	Paradas requieren aprobación admin antes de considerarse entregadas
<code>signature_active</code>	Firma digital obligatoria en entregas

(Verificar alias exactos en BD/settings del enterprise de referencia.)

5.8 Disponibilidad conductor/vehículo

Endpoint `driver_vehicle_availability.php` :

- No asignar driver con ruta activa (status NOT IN completed/finished/cancelled)
- Driver en `workshop` → no disponible para nuevas rutas
- Misma lógica para vehículos

5.9 Encriptación de IDs

Todos los IDs expuestos al frontend/API móvil se **encriptan** con HMAC-SHA256 (`encriptar` / `desencriptar` en `Funciones.php`). Clave: variable de entorno `ENCRYPT_KEY` .

Parámetros API requieren autenticación:

```
tokenAuth(): id_user_over + token → valida contra tabla user
```

5.10 Plantillas vs rutas reales

Concepto	Tabla	Uso
Plantilla	<code>route_template</code> + <code>event_template</code> + <code>stop</code>	Definición reutilizable, sin fechas

Ruta	route + event	Instancia operativa con driver, fechas, status
------	---------------	--

Generación de trayectoria:

- Plantilla: `POST /api/v1/route/template/generate-directions` (NestJS)
- Ruta: `POST /api/v1/route/update-trajectory` (NestJS) o equivalente PHP
- Resultado: `polyline`, `total_distance`, `total_duration`, orden optimizado

5.11 Reportes

- Definidos en tabla `report` con SQL almacenado (`query` field)
- Generación vía `report_generate.php` / `table_generate.php`
- **Importante:** Queries deben filtrar por `COALESCE(r.date_started, r.date_start)` (migración 2026-08-25)
- Reporte clave: `reporte_paradas` — paradas por enterprise/fecha

5.12 Notificaciones

- WhatsApp vía Railway (`WP_BACKEND` , `WP_TOKEN` en `.env` PHP)
- NestJS: endpoint notification (`WP_URL` en `svelte hooks.js`)
- Notificar cliente desde panel cuando ruta está en progreso

6. Flujos operativos principales

6.1 Crear ruta desde admin

1. Seleccionar enterprise → client → driver → vehicle
2. Elegir plantilla o crear manual (wizard multi-paso)
3. Agregar/ordenar paradas (stops)
4. Generar trayectoria (Google Maps Directions)
5. `create_route.php` : insert route, events, checklist_events
6. Sync status driver/vehicle → `enroute` (desde pending)

6.2 Ejecución en campo (app móvil)

1. Login → `user_login.php` → token + `id_user` encriptados
2. Lista rutas del driver filtradas por fecha/status
3. Abrir ruta → ChecklistControl → completar/aprobar
4. Por cada parada: navegar, entregar, evidencia, firma, `deliver_status`
5. Finalizar ruta → status `finished` o auto `completed`

6.3 Dashboard admin

- KPIs: pending, onroute, completed (`dashboard_routes_kpi`)
- Mapa tracking para rutas activas
- Incidencias, notificaciones, carga diferida de widgets pesados

7. Variables de entorno

7.1 rutaflow-app (.env)

```
ENVIRONMENT=development|production
```

```

# MySQL
DB_HOST=
DB_PORT=3306
DB_NAME=rutaflow|rutaflow_dev
DB_USER=
DB_PASSWORD=

# URLs
BASE_HOST=https://app.rutaflow.com
API_HOST=https://app.rutaflow.com
API_BACKEND=https://rutaflow-api-production.up.railway.app

# Seguridad
ENCRYPT_KEY=

# WhatsApp
WP_BACKEND=
WP_TOKEN=

# Google
GOOGLE_TRANSLATE_API_KEY=

# S3/T3 Storage
S3_ENDPOINT=https://t3.storageapi.dev
S3_REGION=auto
S3_BUCKET_PROD=
S3_BUCKET_DEV=
S3_ACCESS_KEY_PROD=
S3_SECRET_KEY_PROD=
S3_ACCESS_KEY_DEV=
S3_SECRET_KEY_DEV=

# Cloudflare Images (fotos conductores)
CLOUDFLARE_ACCOUNT_ID=
CLOUDFLARE_API_TOKEN=

```

Ver plantilla completa en [rutaflow-app/docs/DEPLOYMENT.md](#) .

7.2 rutaflow-api (.env)

```

DATABASE_URL=mysql://user:pass@host:port/db
JWT_SECRET=
GOOGLE_MAPS_API_KEY=
TWILIO_*=
PORT=3001

```

7.3 rutaflow-svelte

```

# Opcional - hooks.js tiene defaults
VITE_WP_TOKEN=

```

```
VITE_WP_URL=
```

`DATABASE_URL` en `src/hooks.js` — cambiar según entorno (prod/dev).

8. Endpoints API principales (PHP)

Base: `{API_HOST}/api/admin/`

Módulo	Ejemplos	Auth
user	<code>user/user_login.php</code>	Público login
route	<code>route/create_route.php</code> , <code>route/delete_route.php</code>	tokenAuth
app	<code>app/driver_routes.php</code> , <code>app/update_event.php</code>	tokenAuth
manager	<code>manager/operations.php</code> , <code>manager/upload_document.php</code>	tokenAuth
dashboard	<code>dashboard/general_dashboard.php</code>	tokenAuth
report	<code>report/report_generate.php</code>	tokenAuth

Patrón request: POST con `id_user_over` , `token` , parámetros encriptados.

NestJS base: `{API_BACKEND}/api/v1/`

9. Guía de replicación paso a paso

Fase 1 — Infraestructura

1. Crear instancia **MySQL 8**
2. Importar schema: `rutaflow-app/api/db/db_170925.sql`
3. Aplicar migraciones en orden: `api/db/migrations/*.sql`
4. Crear buckets S3/T3 (prod + dev)
5. Obtener API keys: Google Maps, Cloudflare Images, WhatsApp

Fase 2 — Backend PHP (núcleo)

```
cd rutaflow-app
cp docs/DEPLOYMENT.md # referencia
# Crear .env con credenciales
php -S localhost:8000 -t .
# Acceder: http://localhost:8000/sys/
```

Requisitos PHP: extensions `mysqli/pdo_mysql`, `curl`, `gd`, `fileinfo`.

Fase 3 — API NestJS (opcional pero recomendado)

```
cd rutaflow-api
npm install
cp .env.example .env # configurar DATABASE_URL
```

```
npx prisma generate
npm run start:dev      # puerto 3001
```

Fase 4 — App móvil

```
cd rutaflow-svelte
npm install
# Editar src/hooks.js → DATABASE_URL apuntando a tu PHP
npm run dev      # http://localhost:5173
npm run build    # producción
npm run build:mobile  # Capacitor Android
```

Fase 5 — Datos semilla

1. Crear enterprise + user admin
2. Crear client + stops + checklist items
3. Crear driver (user type driver) + vehicle
4. Crear route_template → generar directions
5. Crear ruta de prueba → verificar flujo móvil

Fase 6 — Deploy producción

1. **Admin PHP:** servidor con PHP-FPM/Apache, `.env` prod, SSL
2. **Svelte:** build Node adapter → Railway/VPS, dominio driver.*
3. **NestJS:** Railway con DATABASE_URL prod
4. Configurar DNS + Cloudflare
5. Purgar caché CDN tras deploy frontend

10. Checklist de verificación post-replicación

- ☐ Login admin funciona
- ☐ Login conductor app móvil funciona
- ☐ Crear ruta + paradas + polyline
- ☐ Checklist se instancia al crear ruta
- ☐ Conductor ve ruta del día (filtro COALESCE fechas)
- ☐ Completar parada + subir evidencia a S3
- ☐ Foto conductor sube a Cloudflare Images
- ☐ Reporte paradas filtra por fecha correcta
- ☐ Status driver/vehicle se sincroniza al cerrar ruta
- ☐ Encriptación IDs round-trip funciona
- ☐ Notificación WhatsApp (si configurada)

11. Migraciones SQL importantes (orden cronológico)

Archivo	Propósito
2026-04-04_extend_driver_vehicle_img_url.sql	URLs largas imágenes

2026-04-26_sync_route_driver_vehicle_status.sql	Sync status entidades
2026-07-10_add_stop_reference_file.sql	Archivo referencia parada
2026-07-15_add_route_date_started.sql	Campo date_started
2026-07-15_update_reporte_paradas_filters.sql	Filtros reporte
2026-08-16_add_driver_workshop_available_status.sql	Status workshop
2026-08-16_align_vehicle_status_enum.sql	Enum vehicle
2026-08-25_fix_report_date_started_filter.sql	Crítico: WHERE reportes usa date_started

12. Decisiones arquitectónicas y deuda técnica

1. **Dual backend:** PHP (legacy, completo) + NestJS (parcial). Replicar requiere ambos o migrar endpoints NestJS a PHP.
2. **Prisma desactualizado:** No incluye todos los enums/status de prod. No usar solo Prisma para migraciones.
3. **AngularJS legacy:** Panel admin difícil de extender; README raíz describe plan migración Laravel+React.
4. **IDs encriptados:** Cualquier cliente nuevo debe implementar mismo algoritmo HMAC.
5. **Secrets en código:** Mover keys hardcodeadas (hooks.js, S3Config defaults) a variables de entorno.
6. **Multi-tenant:** Siempre filtrar por `id_enterprise` en queries custom.

13. Referencias en el repositorio

Documento	Ubicación
Plan migración tecnológica	/README.md
Deploy admin PHP	rutaflow-app/docs/DEPLOYMENT.md
Endpoints API	rutaflow-app/docs/API_ENDPOINTS.md
Schema Prisma	rutaflow-api/src/prisma/schema.prisma
Dump BD	rutaflow-app/api/db/db_170925.sql
Migraciones	rutaflow-app/api/db/migrations/
Config S3 centralizada	rutaflow-app/api/framework/S3Config.class.php
Lógica status entidades	rutaflow-app/api/framework/Funciones.php
Filtros fecha rutas	rutaflow-app/api/clase/route/Route.class.php

14. Contacto y mantenimiento

- **Autor git configurado:** Roberto-rgb-code
- **Repositorio:** monorepo `rutaflow` con submódulos/proyectos
- **Actualizar este documento** cuando se agreguen migraciones, endpoints o reglas de negocio nuevas.

Documento generado para replicación del sistema Rutaflow. No incluye credenciales; configurar via `.env` en cada entorno.